프로젝트 보고서(총 15점)

학번	3102	성명	김시우
프로젝트명	배고 — 위치 기반 식사 추천 서비스 (Web-view App)		
개발 기간	2026년 3월 6일 - 2026년 4월 25일 (총 8주)		
담당 역할	Frontend (기능/연동) — Geolocation 훅, 전역 상태 관리, 실 API 연동, 딥링크 처리, 즐겨찾기 기능		

1. 개요 및 목표 (1점)

가. 프로젝트 기획 의도

점심·저녁 식사 시 메뉴를 결정하지 못하는 상황은 직장인과 학생 모두가 일상적으로 경험하는 문제다. 기존 네이버 지도, 카카오맵 등의 서비스는 사용자가 검색어를 직접 입력하거나 카테고리를 선택해야 하는 능동적 탐색 방식을 전제로 한다. 이는 메뉴 결정 과정에서 오히려 선택지를 늘려 결정 피로도를 높이는 역효과를 낳는다.

본 프로젝트 **배고**는 HTML5 Geolocation API로 사용자 위치를 자동 감지하고, Naver 지역 검색 API를 프록시 서버로 경유하여 신뢰도 높은 식당 데이터를 즉시 추천하는 **수동적(passive) 추천 서비스**를 구현하는 것을 목표로 한다. 검색어 입력 없이 터치 한 번으로 메뉴를 결정할 수 있는 최소 기능 제품(MVP)을 구축하고, 이후 사용자 선택 이력 데이터를 기반으로 개인화 추천 엔진으로 고도화하는 것을 장기 목표로 설정하였다.

나. 최종 요구사항 및 기능 명세

기능	요구사항	상태
위치 자동 감지	Geolocation API로 위경도 수집, 권한 거부 시 기본 좌표(강남역) fallback	완료
식당 추천	카테고리(한/중/일/양식), 반경(500m/1km/2km) 필터 기반 추천 결과 제공	완료
프록시 서버	Naver API 키 보안을 위한 서버사이드 프록시, CORS 처리	완료
캐싱	Redis로 식당 목록 TTL 30분, 역지오코딩 TTL 24시간 캐싱	완료

딥링크 연동	식당 클릭 시 네이버 지도 앱 또는 모바일 웹으로 직접 이동	완료
즐겨찾기	식당 즐겨찾기 저장 및 목록 조회 (localStorage 기반 MVP)	완료

2. 기능 구현 및 결과 (4점)

가. 전체 시스템 흐름도

사용자 (모바일 브라우저)

↓ HTML5 Geolocation API (위경도 수집)

↓ Zustand 전역 상태 (필터: 카테고리, 반경)

↓ Axios 요청 → NestJS 프록시 서버 (BE)

↓ Redis 캐시 확인 → HIT: 즉시 응답 / MISS: Naver API 호출

↓ Naver Search Local API / Reverse Geocoding API

↓ 응답 가공 후 FE 반환

↓ React 컴포넌트 렌더링 (RestaurantCard, BottomSheet)

↓ 식당 클릭 → 네이버 지도 앱 딥링크 (nmap://) 또는 웹 fallback

[그림 1] 배고 서비스 전체 시스템 데이터 흐름도

나. 주요 기능 구현 내역

기능 1. GPS 위치 감지 커스텀 훅 (useGeolocation)

- 구현 내용: navigator.geolocation.getCurrentPosition을 래핑한 useGeolocation 훅을 제작하였다. 로딩 상태, 에러 객체, 성공 시 위경도(lat, lng)를 반환하며, 권한 거부(PERMISSION_DENIED)와 타임아웃(5초) 상황 모두를 분기 처리하였다.

```
// src/hooks/useGeolocation.ts
export const useGeolocation = () => {
  const [location, setLocation] = useState({ lat: 37.4979, lng: 127.0276 });
  const [error, setError] = useState(null);

  useEffect(() => {
    if (!navigator.geolocation) { setError('Geolocation is not supported');
return; }
    navigator.geolocation.getCurrentPosition(
      (pos) => setLocation({ lat: pos.coords.latitude, lng: pos.coords.longitude
}),
      (err) => {
        setError(err.code === 1 ? 'PERMISSION_DENIED' : 'TIMEOUT');
        // 기본 좌표(강남역) 유지
      },
      { timeout: 5000, maximumAge: 60000 }
    );
  }, []);
```

```
  return { location, error };
};
```

[코드 1] 권한 거부 및 타임아웃 예외 처리가 포함된 useGeolocation 커스텀 훅

- 결과 화면: 위치 감지 성공 시 상단 헤더에 행정동 주소가 표시되고, 권한 거부 시 "기본 위치로 추천합니다" Toast 메시지와 함께 강남역 기준 추천 결과가 노출된다.
- 예외 상황 처리: 권한 거부/타임아웃 어느 경우에도 앱이 로딩 화면에 멈추지 않고 기본 좌표로 서비스를 정상 제공한다.

기능 2. Zustand 전역 상태 관리 및 필터 연동

- 구현 내용: 사용자가 선택한 카테고리·반경 필터를 Zustand store(useFilterStore)로 전역 관리하여, BottomSheet에서 필터 변경 시 useRestaurants 훅이 새 파라미터로 API를 자동 재호출하는 단방향 데이터 흐름을 구성하였다.
- 예외 상황 처리: store 초기화 타이밍과 API 호출 타이밍이 어긋나는 race condition을 방지하기 위해 파라미터 유효성 검사 후 기본값 fallback 처리를 useRestaurants 훅 내부에 추가하였다.

기능 3. Axios Interceptor 및 통합 에러 처리

- 구현 내용: Axios 인스턴스에 Request/Response Interceptor를 적용하여 공통 헤더 자동 주입 및 HTTP 상태 코드별 에러 처리를 중앙화하였다. 네트워크 단절, 5xx 서버 오류, 429 Rate Limit 초과 상황마다 적절한 Toast 메시지를 노출한다.

```
// src/lib/axiosInstance.ts
axiosInstance.interceptors.response.use(
  (response) => response,
  (error) => {
    if (!error.response) {
      toast.error('인터넷 연결을 확인해주세요.');
    } else if (error.response.status >= 500) {
      toast.error('서버 오류가 발생했습니다. 잠시 후 다시 시도해주세요.');
    } else if (error.response.status === 429) {
      toast.error('요청이 너무 많습니다. 잠시 후 다시 시도해주세요.');
    }
    return Promise.reject(error);
  }
);
```

[코드 2] HTTP 상태 코드별 통합 에러 처리가 적용된 Axios Response Interceptor

- 예외 상황 처리: 각 컴포넌트에서 개별 try-catch를 작성하지 않아도 모든 API 오류가 인터셉터에서 일괄 처리되어 에러 처리 누락을 구조적으로 방지한다.

기능 4. 네이버 지도 딥링크 연동

- 구현 내용: 식당 카드 클릭 시 openNaverMap 유틸 함수가 네이버 지도 앱(nmap://) URL을 먼저 시도하고, 500ms 타임아웃 후 앱이 없으면 모바일 웹으로 fallback한다. radius 단위 오해(m↔km) 버그를 백엔드 담당자와 명세 재확인 후 변환 로직(radius / 1000)을 추가하여 해결하였다.
- 예외 상황 처리: 앱 미설치 환경에서도 웹 브라우저로 정상 연결되어 서비스 단절 없이 길 찾기를 제공한다.

3. 서비스 연동 및 데이터 처리 (2점)

가. 외부 서비스 연동 내역

서비스	용도	연동 방식
Naver Search Local API	위치 기반 식당 목록 조회	BE 프록시 서버 경유, Client-ID/Secret 서버사이드 .env 관리
Naver Reverse Geocoding API	위경도 → 행정동 주소 변환	BE 프록시 서버 경유, 결과 Redis 24시간 캐싱
Redis 7.x	API 응답 캐싱, Rate Limit 카운터	ioredis 클라이언트, Docker 컨테이너로 BE와 함께 구동
Naver 지도 딥링크	식당 상세 정보 및 길 찾기 연동	nmap:// 앱 스킴 + map.naver.com 웹 fallback
AWS EC2 (t3.micro)	백엔드 서버 프로덕션 배포	Docker Compose, GitHub Actions CI/CD 자동 배포

나. 데이터 처리 방법

FE에서 useGeolocation 훅이 위경도를 수집하면 Zustand store의 필터 상태(카테고리, 반경)와 함께 Axios 인스턴스를 통해 BE 프록시 서버로 전달된다. BE(NestJS)는 요청 수신 즉시 ValidationPipe로 파라미터를 검증하고, CacheService를 통해 Redis에서 캐시 키(lat.toFixed(3):lng.toFixed(3):category:radius)를 조회한다. 캐시 HIT 시 Redis 데이터를 즉시 반환하며, MISS 시 Naver Search Local API를 호출하여 결과를 Redis에 저장(TTL 30분) 후 반환한다. FE는 수신한 식당 목록을 RestaurantCard 컴포넌트에 바인딩하여 렌더링한다. 즐겨찾기 데이터는 MVP 단계에서 localStorage에 저장되며, 추후 BE /api/favorites 엔드포인트와 동기화할 예정이다.

4. 유지보수 계획 (2점)

가. 주요 핵심 코드 설명

```
// src/utils/openNaverMap.ts
export const openNaverMap = (name: string, lat: number, lng: number) => {
  const appUrl = `nmap://place?
lat=${lat}&lng=${lng}&name=${encodeURIComponent(name)}&appname=com.baego.app`;
  const webUrl = `https://map.naver.com/v5/search/${encodeURIComponent(name)}?
c=${lng},${lat},15,0,0,0,dh`;

  window.location.href = appUrl;

  setTimeout(() => {
    window.location.href = webUrl; // 앱 미설치 시 웹으로 fallback
  }, 500);
};
```

[코드 3] 네이버 지도 앱 딥링크 및 500ms 타임아웃 기반 웹 fallback 유틸리티 함수

위 함수는 nmap:// 스킴으로 앱 실행을 먼저 시도한다. 앱이 설치된 경우 즉시 이동되고 setTimeout의 webUrl 이동은 실행되지 않는다. 앱이 없는 경우 스킴 처리가 실패하고 500ms 후 모바일 웹으로 fallback되어 어떤 환경에서도 길 찾기를 제공한다.

나. 유지보수 계획

항목	현황	개선 계획
즐겨찾기 동기화	localStorage 기반 MVP	/api/favorites 엔드포인트와 device_id 기반 서버 동기화로 기기 간 데이터 유지
API 상태 관리	커스텀 훅 + Axios 인스턴스	React Query(TanStack Query) 도입으로 캐싱·리패치·로딩 상태 선언적 관리
API 명세 관리	구두 협의 + 수동 업데이트	OpenAPI(Swagger) 스펙 기반 자동 타입 생성으로 FE/BE 간 타입 불일치 방지
딥링크 fallback	500ms 고정 타임아웃	document.hidden 이벤트 기반 앱 전환 감지로 fallback 정확도 개선
선택 이력 연동	미구현	/api/history 엔드포인트 연동으로 선택 이력 서버 저장 및 개인화 추천 기반 마련

5. 결론 및 고찰 (4점)

가. 최종 시연 결과

프론트엔드 기능/연동 담당으로서 위치 감지(useGeolocation), 전역 상태 관리(Zustand), 실 API 연동(Axios Interceptor), 딥링크 처리(openNaverMap), 즐겨찾기(localStorage) 등 서비스의 핵심 기능 동작을 모두 구현하였다. Mock 데이터 기반 UI 바인딩 검증 이후 실제 백엔드 API로의 전환도 타입 정의 수정을 기점으로 컴파일 오류를 따라가는 방식으로 신속하게 완료하였다. 실 API 전환 후 radius 단위 오해로 발생한 빈 결과 버그를 백엔드 담당자와 명세 재확인을 통해 해결한 경험은, FE-BE 간 API 명세를 코드 작성 전에 명확히 합의하는 것의 중요성을 실증적으로 보여주었다.

나. 고찰

FE 기능/연동 담당으로서 이번 프로젝트를 통해 UI와 데이터 로직을 분리하는 Custom Hook 패턴의 실용성을 체감하였다. useGeolocation, useRestaurants, useFilterStore와 같이 기능 단위로 로직을 분리함으로써 UI 컴포넌트는 렌더링에만 집중하고, 상태 변화와 API 호출은 훅 내부에서 독립적으로 관리할 수 있었다. 이 구조 덕분에 Mock 데이터에서 실 API로 전환할 때 컴포넌트 코드를 변경하지 않고 훅 내부만 수정하여 교체가 가능하였다.

반면 Zustand store 초기화 타이밍과 API 호출 타이밍이 맞지 않아 radius 파라미터가 undefined로 전달되던 race condition 문제는, 비동기 상태 관리의 타이밍 이슈를 사전에 설계 단계에서 고려해야 함을 상기시켜 주었다. 각 컴포넌트에서 개별적으로 작성하던 try-catch를 Axios Interceptor로 중앙화한 후 에러 처리 누락 케이스가 사라진 경험도 인상적이었다.

심화 연구 방향으로는 첫째, 현재 커스텀 훅 기반의 API 상태 관리를 React Query(TanStack Query)로 전환하여 캐싱, 리패치, 백그라운드 갱신을 선언적으로 처리하고 불필요한 API 중복 호출을 줄이는 방향을 검토할 수 있다. 둘째, 즐겨찾기 데이터를 /api/favorites 엔드포인트와 동기화하고 선택 이력(/api/history)을 서버에 누적하여 카테고리별 선호도 기반 개인화 추천 가중치를 적용하는 것이 서비스의 장기적 차별화 포인트가 될 수 있다.